

EE-2003

Computer Organization and Assembly Language

Week 04 – Lecture 08

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

September 9, 2025

Outline

- 1 Quick Review
- 2 Clearing the AX Register
- 3 Register Indirect Addressing (iAPX 88)
- 4 Loops and Program Control in Assembly
- 5 Register + Offset Addressing (Concept)
- 6 Segment Association (Concept)
- 7 Address Wraparound

Quick Review: What is Done in Last lecture?

- Memory Addressing
- CS & DS Registers
- Working on these registers directly in DosBox
- Why Segmentation is important?

Using db Instead of dw (Problem)

Problem when mixing db with 16-bit registers:

```
mov  ax, [num1]      ; tries to load word (16-bit)
mov  bx, [num1+1]
add  ax, bx
mov  bx, [num1+2]
add  ax, bx
mov  [num1+3], ax

mov  ax, 0x4c00
int  0x21
```

```
num1: db  5, 10, 15, 0
```

- db reserves 1 byte per value, but AX/BX are 16-bit registers.
- This leads to **garbage values and misalignment**.
- Example: instead of loading 5, the CPU may combine two adjacent bytes (e.g., 0A05).

Using db Instead of dw (Solution)

Solution: match register size with data size.

```
mov  al, [num1]
mov  bl, [num1+1]
add  al, bl
mov  bl, [num1+2]
add  al, bl
mov  [num1+3], al
```

```
mov  ax, 0x4c00
int  0x21
```

```
num1: db  5, 10, 15, 0
```

- Use **8-bit registers** (AL, BL) with db.
- Each variable fits exactly into one byte.
- Correct results, no mismatch or garbage values.
- Best for small integers (0–255).

Clearing the AX Register

- Sometimes the AX register contains an **old value** from previous instructions.
- Before reusing AX, we must **clear it to 0** to avoid incorrect results.
- Example situation: performing addition, multiplication, or string operations where leftover data can corrupt the result.

Two Common Ways to Clear AX

- `mov ax, 0` (direct assignment to 0)
- `xor ax, ax` (XOR with itself → always 0)
- `xor ax, ax` is **more efficient**:
 - Smaller instruction encoding (1 byte shorter).
 - Executes faster on most CPUs.
 - Does not require fetching an immediate constant from memory.

Direct vs Indirect Addressing in Assembly

Example Instructions

- `mov dx, 0119`
`mov bx, [0119]`
- `mov bx, [num1]` ,

→ **Indirect Addressing**

→ **Direct Addressing**

✓ Pros

- **Direct:** Easy to read and understand, clear labels.
- **Indirect:** Flexible, address can change dynamically using registers.

× Cons

- **Direct:** Harder to reuse , tied to fixed memory labels.
- **Indirect:** Less readable, requires extra setup of registers.

Takeaway: Direct = simpler and static, Indirect = powerful and dynamic.

Register Indirect Addressing (iAPX 88)

- Direct addressing fixes the memory location in the instruction itself.
- Problem: Adding many numbers (e.g., 100) would require **hundreds of instructions**.
- Solution: Use a **register to hold the address**, and update it to access consecutive memory cells.
- In iAPX 88, the registers that can hold addresses are: **BX, BP, SI, and DI**.
- Addressing uses [register] form:
 - `mov ax, [bx]` → Move into AX the contents of memory at address in BX.
 - Without brackets (`mov ax, bx`) → Moves the value of BX, not memory contents.
- This makes the data access **dynamic, flexible, and programmable**.

Example: Adding Numbers with Indirect Addressing

Assembly Code (adding 3 numbers)

```
mov bx, num1      ; point BX to first number
mov ax, [bx]       ; load first number in AX
add bx, 2          ; move to second number
add ax, [bx]       ; add second number
add bx, 2          ; move to third number
add ax, [bx]       ; add third number
add bx, 2          ; move to result cell
mov [bx], ax       ; store sum at num1+6
num1: dw 5, 10, 15, 0
```

- BX initially contains the **address of num1**. [BX] accesses the contents at that memory address.
- Adding 2 to BX (word size) moves it to the **next number in memory**.
- This mechanism allows the same instruction form to process many **numbers efficiently**.

Loops and Program Control in Assembly

- Adding many numbers with direct instructions is inefficient (100 numbers = 200 instructions).
- We need:
 - ① A way to **change the address** (done using registers like BX).
 - ② A way to **repeat instructions** until a terminal condition is met.
- New instructions introduced:
 - SUB → subtraction with flags update.
 - JNZ (Jump if Not Zero) → conditional jump based on Zero Flag.
- Together, these allow us to build **loops** that repeat tasks efficiently.

Example: Adding 10 Numbers with a Loop

Assembly Program

```
mov bx, num1      ; point BX to first number
mov cx, 10         ; load count in CX
mov ax, 0          ; initialize sum to zero

l1: add ax, [bx]    ; add number to AX
    add bx, 2       ; move BX to next word
    sub cx, 1       ; reduce counter
    jnz l1          ; repeat if CX != 0

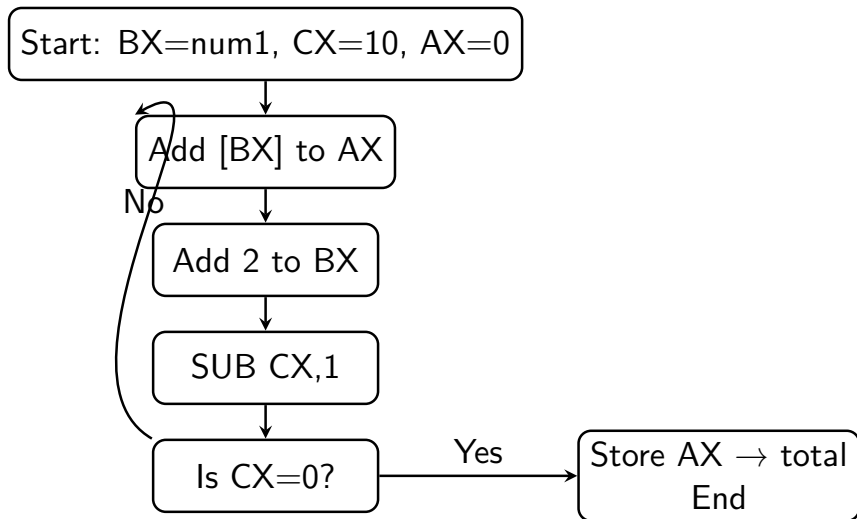
mov [total], ax    ; store result
num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0
```

- BX → address of current number.
- CX → loop counter, decremented each step.
- AX → accumulator for the sum.

Understanding the Loop Execution

- SUB CX,1 clears Zero Flag the first 9 times, sets it on the 10th.
- JNZ l1 jumps back to label l1 if ZF=0, else continues.
- This creates a **count-controlled loop**.
- Important observations:
 - Labels (e.g., l1) are just addresses – assembler doesn't distinguish code vs data.
 - Jump loads label address into IP → program flow changes.
 - BX/BP used → called **based addressing**.
 - SI/DI could also be used → called **indexed addressing**.
- Conditional jumps (JNZ) give the CPU **decision-making capability**.

Flow of Program Execution (Loop)



Maximum Loop Execution!

- The CX register is used as a loop counter in 8086/8088.
- Since CX is 16-bit, it can store values from 0000h to FFFFh.
- Maximum starting value = FFFFh = 65,535₁₀.
- Therefore, a loop controlled by CX can execute at most:

65,535 times

- This makes CX suitable for repetitive tasks such as string operations and repeated arithmetic.

Importance of Flag Register (Zero Flag)

- The **Flag Register** stores status information about the result of operations.
- The **Zero Flag (ZF)** is set when the result of an operation becomes 0.
- In loops, ZF plays a critical role:
 - If $ZF = 0 \rightarrow$ the loop continues.
 - If $ZF = 1 \rightarrow$ the loop ends.
- Example: After each `SUB CX, 1`, ZF is checked by `JNZ`.

AX 001E	SI 0000	CS 19F5	IP 0117	Stack +0 0000	Flags 7200
BX 0125	DI 0000	DS 19F5		+2 20CD	
CX 0027	BP 0000	ES 19F5	HS 19F5	+4 9FFF	OF DF IF SF ZF AF PF CF
DX 0000	SP FFEE	SS 19F5	FS 19F5	+6 EA00	0 0 1 0 0 0 0 0

CMD > █		0000		
0113	81C30200	ADD	BX,0002	
0117	A32501	MOV	[0125],AX	
011A	B8004C	MOV	AX,4C00	
011D	CD21	INT	21	
011F	05000A	ADD	AX,0A00	
0122	000F	ADD	[BX],CL	
0124	0000	ADD	[BX+SI],AL	
0126	00E0	ADD	AL,AH	
0128	D1E0	SHL	AX,1	

	3	4	5	6	7	8	9	A
DS:0123	0F	00	00	00	E0	D1	E0	C5
DS:012B	5E	D8	01	C3	8B	07	8B	57
DS:0133	02	85	D2	75	04	85	C0	74
DS:013B	1C	C7	46	DC	00	00	8E	5E
DS:0143	FC	83	7D	0E	00	74	09	8B
DS:014B	46	F2	48	3B	46	F6	7E	08
DS:0153	B8	01	00	EB	05	E9	42	01
DS:015B	31	C0	89	46	E2	8B	46	F6
DS:0163	D1	E0	D1	E0	C5	5E	D8	01
DS:016B	C3	8B	37	8B	47	02	89	46

2.5 Register + Offset Addressing (Concept)

- Extends direct and indirect addressing by combining both.
- BX holds the **index**, not the full address.
- Effective address is formed at runtime:

$$EA = \text{Base Address (num1)} + BX (\text{index})$$

- Since elements are **words**, BX increments in steps of 2 (0, 2, 4, ...).
- This scheme is called **Base + Offset** (BX/BP) or **Index + Offset** (SI/DI).

2.5 Register + Offset Addressing (Example 2.8)

Program to Add Ten Numbers

```
[org 0x0100]
mov bx, 0          ; initialize array index
mov cx, 10         ; count = 10
mov ax, 0          ; sum = 0

l1: add ax, [num1+bx] ; add number
    add bx, 2        ; step to next word
    sub cx, 1
    jnz l1            ; repeat if CX != 0

mov [total], ax
mov ax, 0x4c00
int 0x21

num1: dw 10,20,30,40,50,10,20,30,40,50
total: dw 0
```

2.6 Segment Association (Concept)

- Effective Address (EA) = result of calculation (e.g., Base + Offset).
- EA is only an **offset**, not the real memory location.
- To get the **Physical Address (PA)**, EA must be combined with a **Segment Base**.
- Formula:

$$PA = (\text{Segment} \times 16) + EA$$

Default Segment Associations

- Different registers have default segment associations:
 - **CS** (Code Segment) → tied to IP (Instruction Pointer).
 - **DS** (Data Segment) → default for BX, SI, DI.
 - **SS** (Stack Segment) → default for SP and BP.
 - **ES** (Extra Segment) → can be used for extra data.
- Some registers (CS–IP, SS–SP) are locked and cannot be changed.
- Others (BX, SI, DI, BP) can override with segment prefixes.

Example: Segment + Effective Address

Instruction: `mov ax, [cs:bx+si+0700h]`

- Given: $BX = 0100h$, $SI = 0200h$, $CS = 1000h$.
- Step 1: Effective Address (EA) = $BX + SI + 0700h$

$$EA = 0100h + 0200h + 0700h = 0A00h$$

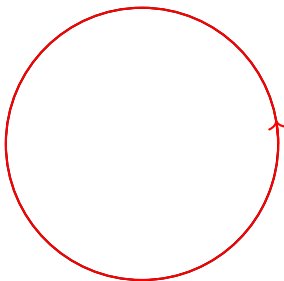
- Step 2: Segment Base = $CS \times 16 = 1000h \times 10h = 10000h$
- Step 3: Physical Address = Segment Base + EA

$$PA = 10000h + 0A00h = 10A00h$$

2.7 Address Wraparound (Concept with Diagram)

- Occurs when address calculations go beyond limits.
- Carry is **dropped**, so addressing restarts from the beginning.
- Two types:
 - **Segment Wraparound** (within a segment).
 - **Physical Wraparound** (within whole memory).

Addresses behave like a circle

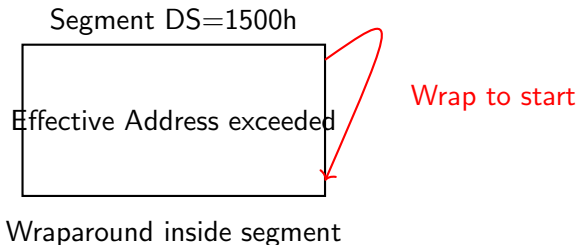


Wraparound $\Rightarrow$ back to start

Segment Wraparound Example

Given: BX=9100h, DS=1500h, access=[bx+7000h]

- $EA = 9100h + 7000h = 10100h$
- Carry dropped $\rightarrow EA = 0100h$
- Physical Address = $1500h \times 10h + 0100h = 15100h$



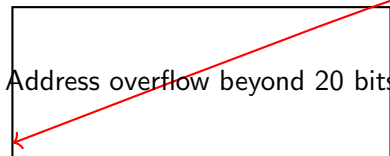
Physical Memory Wraparound Example

Given: BX=0100h, DS=FFF0h, access=[bx+0100h]

- EA = 0200h
- Segment Base = FFF00h
- PA = 100100h (21 bits) → too large for 20-bit bus
- Carry dropped → Actual PA = 00100h

Wraparound at top of memory

1 MB Physical Memory



Wrap to 00000h start of memory

Thank You!

Any Questions?

- We explored: **addressing modes, loops, flags, and segmentation.**
- Key lessons:
 - Match data size with register size (db vs dw).
 - Indirect and indexed addressing enable flexible data access.
 - CX + Zero Flag → loop control and program flow.
 - Segments + offsets form physical addresses (with possible wraparound).
- Takeaway: **Efficient assembly depends on understanding addressing + flags.**

“Master addressing and flags, and you master assembly programming.”